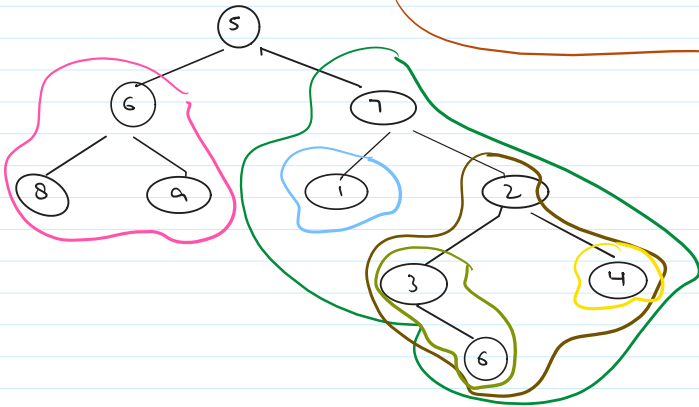
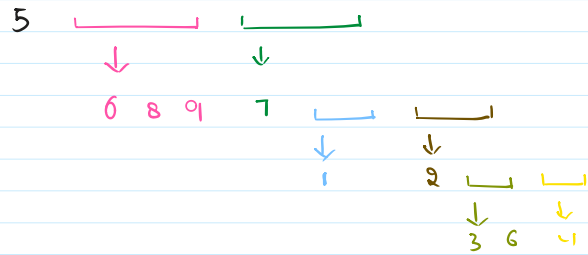


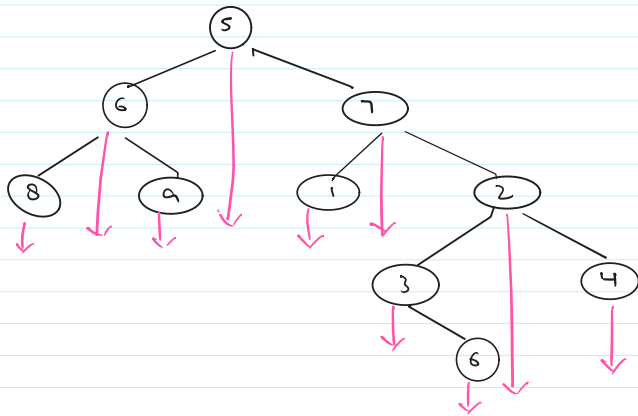
Pre-order Traversal



Root → Left → Right



5 6 8 9 7 1 3 4 5 6



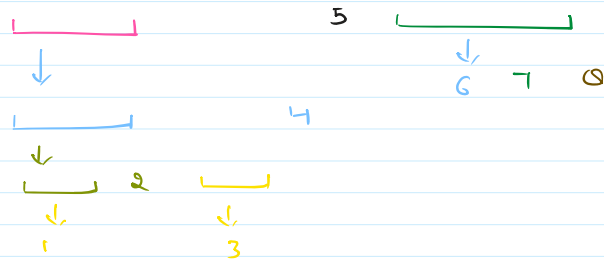
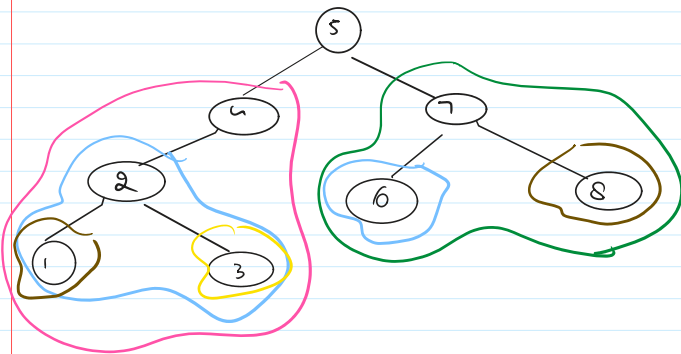
Inorder Traversal

Left ⇒ Root ⇒ Right

8 6 9 5 1 7 3 6 2 4

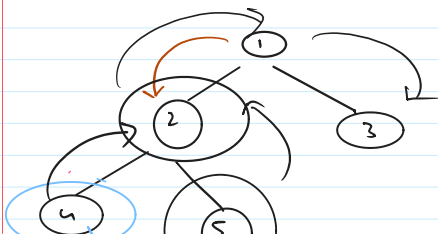
8 6 9 5 1 7 3 6 2 4

Left ⇒ Root ⇒ Right

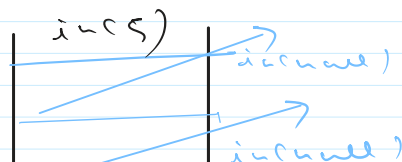


Sorted Traversal

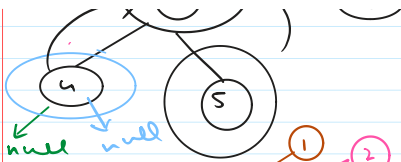
Binary Search Tree



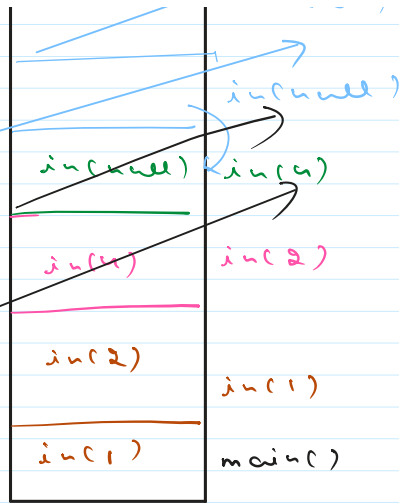
Left ⇒ Root ⇒ Right



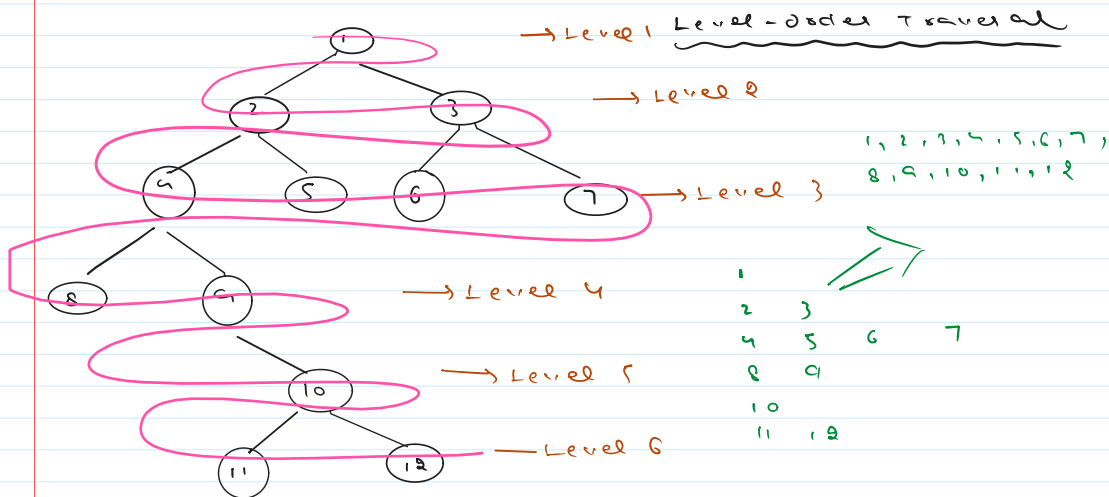
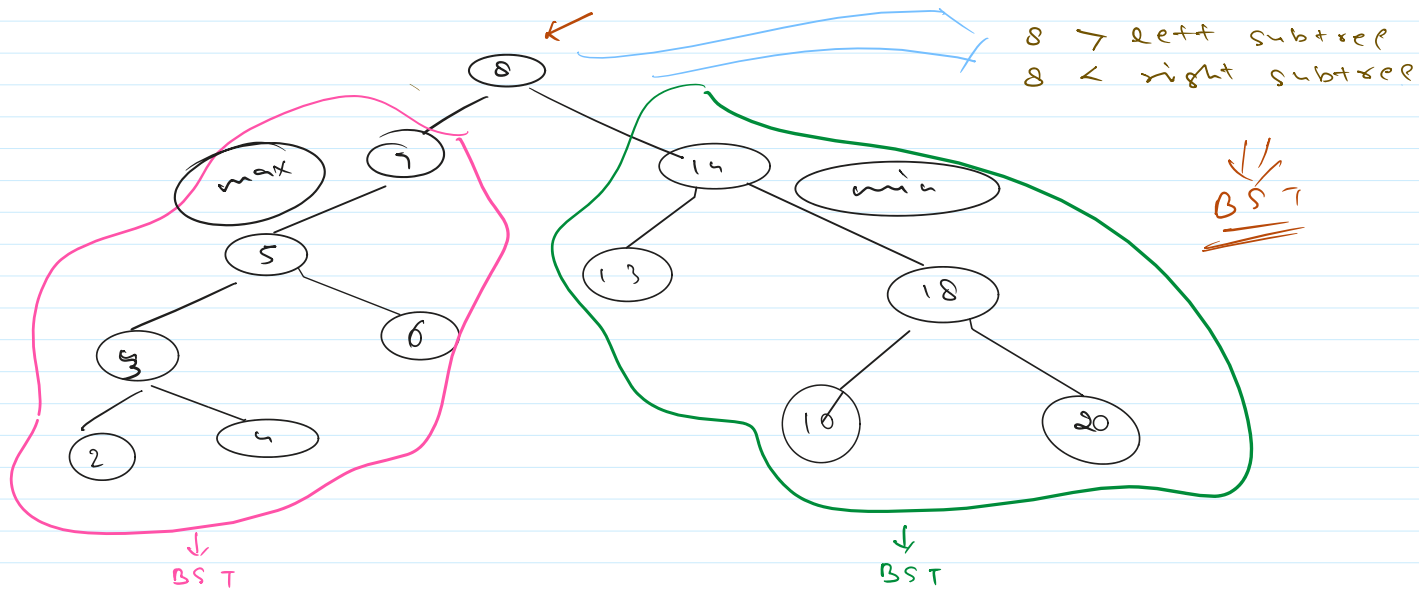
4 2 5 1 3



```
private void inOrder(Node root)
{
    if(root == null) {
        return;
    }
    inOrder(root.left);
    System.out.print(root.val + " ");
    inOrder(root.right);
}
```

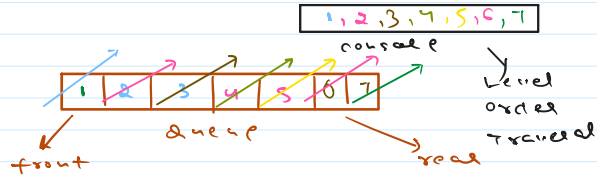
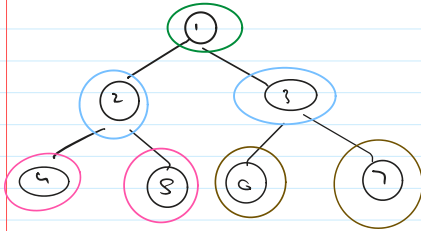
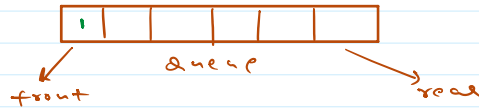
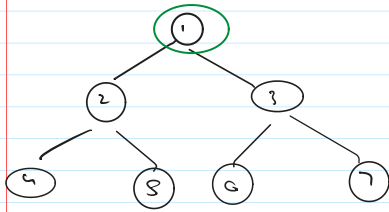


sorted order



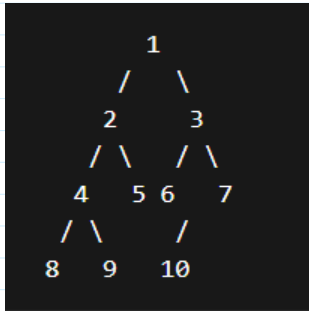
BFT $\Rightarrow$ Breadth First Traversal $\Rightarrow$ Queue
 DFT $\Rightarrow$ Depth First Traversal $\Rightarrow$ Stack





- Till queue is not empty :
1. Pop from queue
 2. Print popped element.
 3. Add left child if exists
 4. Add RC in Q

Queue<node> q = new LinkedList<>(); $\rightarrow$ q.poll() $\Rightarrow$ pop from start



1, 2, 3, 4, 5, 6, 7, 8, 9, 10 X

1
2 3
4 5 6 7
8 9 10

